1/6

Your Containers are running...

NEERAJ SINGH
DEVOPS

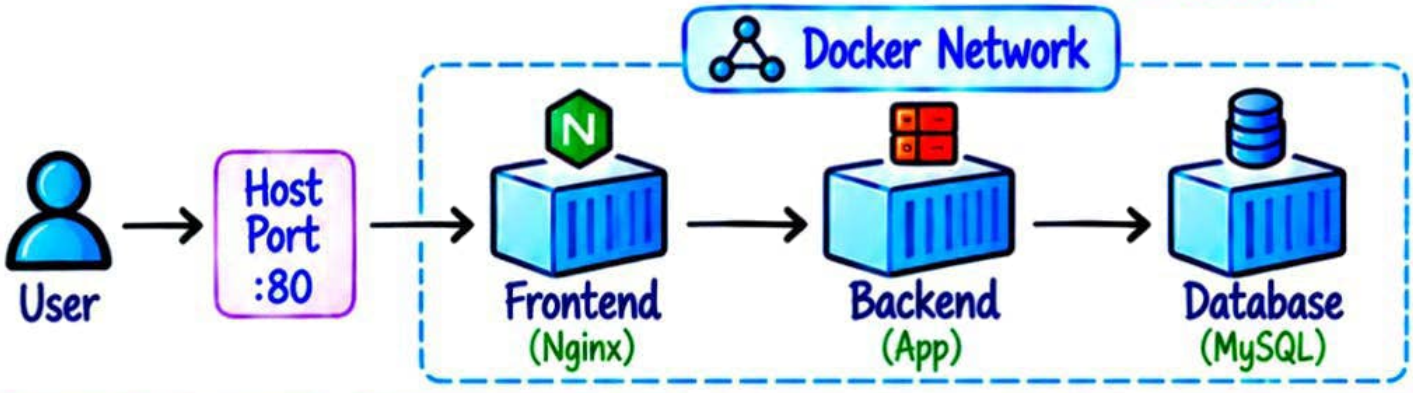
Let's Understand Docker Networking Step by Step !

DOCKER NETWORKING

How Containers Communicate ?

Your containers are running...
but they can't talk to each other.

WHY ?



Bridge
Container Communication

DNS
Use Container Names

Ports
Expose to Host

Secure
Network Isolation

Key Takeaway : CONNECT → COMMUNICATE → EXPOSE → SECURE

Follow | Share | Save

2/6

Different Ways Containers Can Communicate !

Same Host, Same Network, Easy Communication !

HOW DO CONTAINERS COMMUNICATE ?

Docker provides networking so containers can communicate with each other, with the host, and with the external world.

1 Container → Container

Containers on the same Docker network can communicate using container names.

Docker Network (app-network)

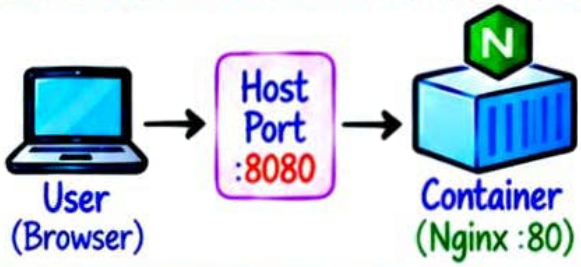


Frontend can call → http://backend:8080
Backend can call → database:3306

2 Host → Container

Use port mapping to access a container from the Docker host.

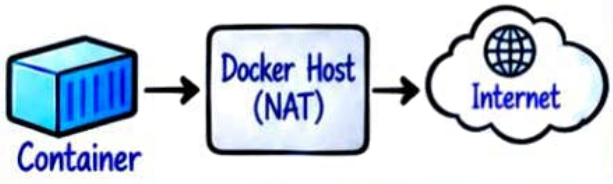
```
docker run -d -p 8080:80 nginx
```



- Host port 8080 is mapped to container port 80
- Now you can access → http://localhost:8080

3 Container → Internet

Containers can access the internet through the Docker host (NAT).



Examples:
- Install packages
- Download dependencies
- Access external APIs

Key Takeaways

1. Containers communicate using Docker networks.
2. Use container names (DNS).
3. Use port mapping to access from host.
4. Containers can reach the internet through the host.
5. Networking is the communication layer between containers.

Docker Network = Communication Layer Between Containers

3/6

Different Network Types for Different Use Cases !

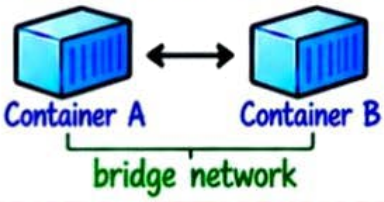
Choose the Right Network for Your Use Case !

DOCKER NETWORK DRIVERS

Docker provides different network drivers to support various communication needs.

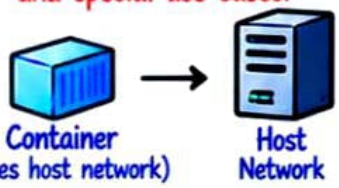
1 BRIDGE

- Default network for containers on a single Docker host.
- Containers on the same bridge network can communicate.
- Uses NAT for external access.
- Best for single-host applications.



2 HOST

- Container uses the host's network directly.
- No network isolation.
- No need for port mapping (-p).
- Useful for high performance and special use cases.



3 NONE

- Disables all networking for the container.
- Container can only use localhost (loopback).
- Useful for network-isolated workloads (more secure).



4 OVERLAY

- Connects containers across multiple Docker hosts.
- Commonly used with Docker Swarm.
- Creates a single logical network across different hosts.
- Used for multi-host / clustered applications.



5 MACVLAN

- Gives containers their own MAC address.
- Container appears as a device directly on the physical network.
- Useful for legacy systems and network-centric applications.



When to Use Which ?

1. **Bridge** – Single-host applications (Default)
2. **Host** – High performance or special use case
3. **None** – Network isolation
4. **Overlay** – Multi-host (Docker Swarm)
5. **Macvlan** – Direct access to physical network

Useful Commands

```
# List networks
docker network ls

# Inspect a network
docker network inspect <name>

# Create a network
docker network create <name>

# Remove a network
docker network rm <name>
```

Key Takeaway : Choose the right network driver based on your application's architecture and communication requirements.

4/6

Lets Create Our Own Network and Connect Containers !

Containers Communicate using Names, Not IPs !

USER-DEFINED BRIDGE NETWORK

User-defined bridge networks provide better isolation and automatic DNS-based container name resolution.

1 Create a Bridge Network

```
$ docker network create app-network
```

2 Run Containers on the Network

```
$ docker run -d \
  --name database \
  --network app-network \
  mysql:8
```

Database container

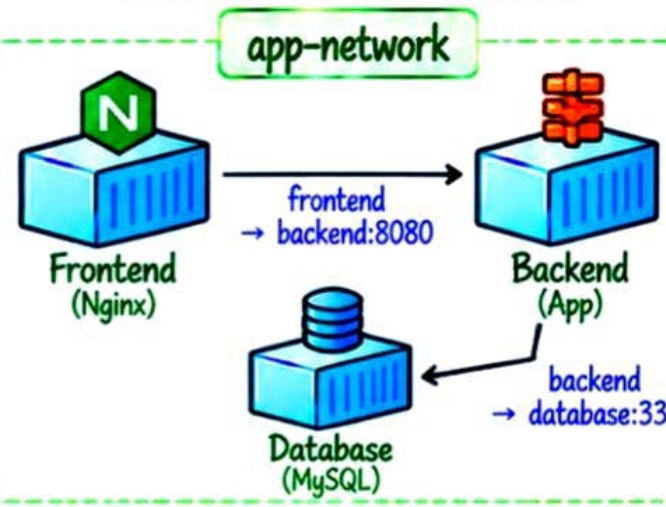
```
$ docker run -d \
  --name backend \
  --network app-network \
  my-backend:latest
```

Backend container

```
$ docker run -d \
  --name frontend \
  --network app-network \
  my-frontend:latest
```

Frontend container

Network Architecture



3 Test Communication

```
$ docker exec -it frontend ping backend
  # frontend can reach backend
```

```
$ docker exec -it backend ping database
  # backend can reach database
```

Use Container Names

- ✓ backend:8080
- ✓ database:3306
- ✗ Don't use IPs 172.x.x.x

Inspect the Network

```
$ docker network inspect app-network
```

List All Networks

```
$ docker network ls
```

Key Takeaways :

1. Create a user-defined bridge network.
2. Run all related containers on the same network.
3. Use container names to communicate.
4. No need to use container IPs.
5. Automatic DNS-based name resolution.
6. Better isolation for your application.

5/6

Expose the Right Ports Keep it Secure !

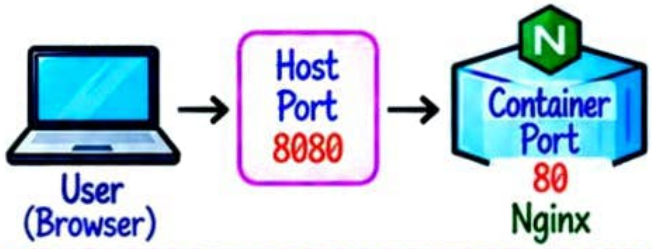
Let External Users Access Your App from the Host !

PORT MAPPING EXPOSE vs -P

Port mapping allows traffic from the Docker host to reach a container.

1 What is Port Mapping ?

- Maps a port on the Docker host to a port inside the container.
- Allows external users to access the application running in the container.
- Syntax : -p HOST_PORT:CONTAINER_PORT




```
$ docker run -d -p 8080:80 nginx
```

2 Example

```
$ docker run -d -p 8080:80 nginx
```

- Host port 8080 is mapped to container port 80.
- Now you can access the application from your host browser using → http://localhost:8080
- You can map to a different host port as well.
- Example: -p 8081:80, -p 3000:3000 etc.



3 EXPOSE vs -p

EXPOSE (in Dockerfile)

```
EXPOSE 80
```

- Documents the port the application listens on.
- Does NOT publish the port to the Docker host.
- Used as metadata / documentation.

-p (during docker run)

```
docker run -p 8080:80 nginx
```

- Publishes the container port to the host.
- Routes traffic from host port to container port.
- Allows external access from browser or other clients.

4 Important Points

- You can map multiple ports: -p 8080:80 -p 443:443
- Use only required ports for security.
- Container-to-container communication does NOT require port mapping.
- EXPOSE does not publish the port.
- You can bind to a specific host IP: -p 127.0.0.1:8080:80
- Check mapped ports using: docker ps

Example: Multiple Port Mapping

```
$ docker run -d -p 8080:80 -p 443:443 nginx
```

Key Takeaway: -p HOST_PORT:CONTAINER_PORT → Exposes the container application to the Docker host and allows external access.

6/6

Build Secure and Reliable Container Networks !

Follow Best Practices for a Secure Network !

DOCKER NETWORKING CHECKLIST & BEST PRACTICES

Keep your Docker networks secure, organized and easy to maintain.

1 Security Best Practices

- Use user-defined networks.
- Segment frontend, backend and database.
- Publish only required ports.
- Don't expose databases to the internet.
- Use container names instead of IPs.
- Avoid hardcoded IP addresses.
- Protect credentials separately.
- Keep Docker and base images updated.
- Use firewall / security groups on host.
- Use least privilege access.

2 Common Issues & Fixes

Containers cannot communicate	Check if they are on the same network.
Application not accessible from host	Verify port mapping with docker ps.
Using localhost inside a container	Use container name (e.g. database) instead of localhost.
Hardcoded container IPs	Use Docker DNS (container names).
Cannot access internet from container	Check network driver and host NAT.

3 Useful Commands

```
# List all networks
docker network ls

# Inspect a network
docker network inspect app-network

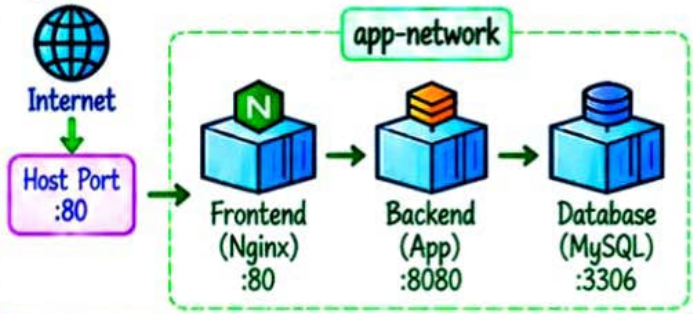
# Create a network
docker network create app-network

# Connect a container to a network
docker network connect app-network <container>

# Disconnect a container
docker network disconnect app-network <container>

# Remove a network
docker network rm app-network
```

4 End-to-End Architecture (Example)



5 Interview Questions

1. What is Docker networking ?
2. What are the network drivers in Docker ?
3. Difference between EXPOSE and -p ?
4. Why use a user-defined bridge network ?
5. Can containers communicate without port mapping ?

6 Key Takeaways

1. Docker network is the communication layer.
2. Use the right driver for your use case.
3. Use container names (Docker DNS).
4. Publish only necessary ports.
5. Follow security best practices.
6. Plan network segmentation for real applications.

Final Takeaway : CREATE → CONNECT → DISCOVER → PUBLISH → SECURE